

U3 · Aula 6 — Pipeline IA Completo, Self-Healing e AI Agents

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Qualidade Web e Mobile (EAD)

Roteiro da aula

- **Parte 1 — Pipeline completo:** geração → execução → healing loop → Healenium
- **Parte 2 — Ecossistema de agentes:** synthetic data, Browser Use, Computer Use
- **Parte 3 — Prática guiada:** seu healing loop no lab IA

Depois desta aula: screencast — eu completando o healing loop passo a passo.

Objetivos de aprendizagem

- **Desenhar** o pipeline IA completo (gen → run → heal → report)
- **Implementar** um healing loop com máximo de tentativas e auditoria
- **Avaliar** self-healing criticamente — quando mascara regressão real
- **Comparar** agentes (Browser Use, Computer Use) por custo e valor
- **Concluir** o lab IA — pipeline funcional de ponta a ponta

Parte 1 — Pipeline completo

Geração → execução → healing loop

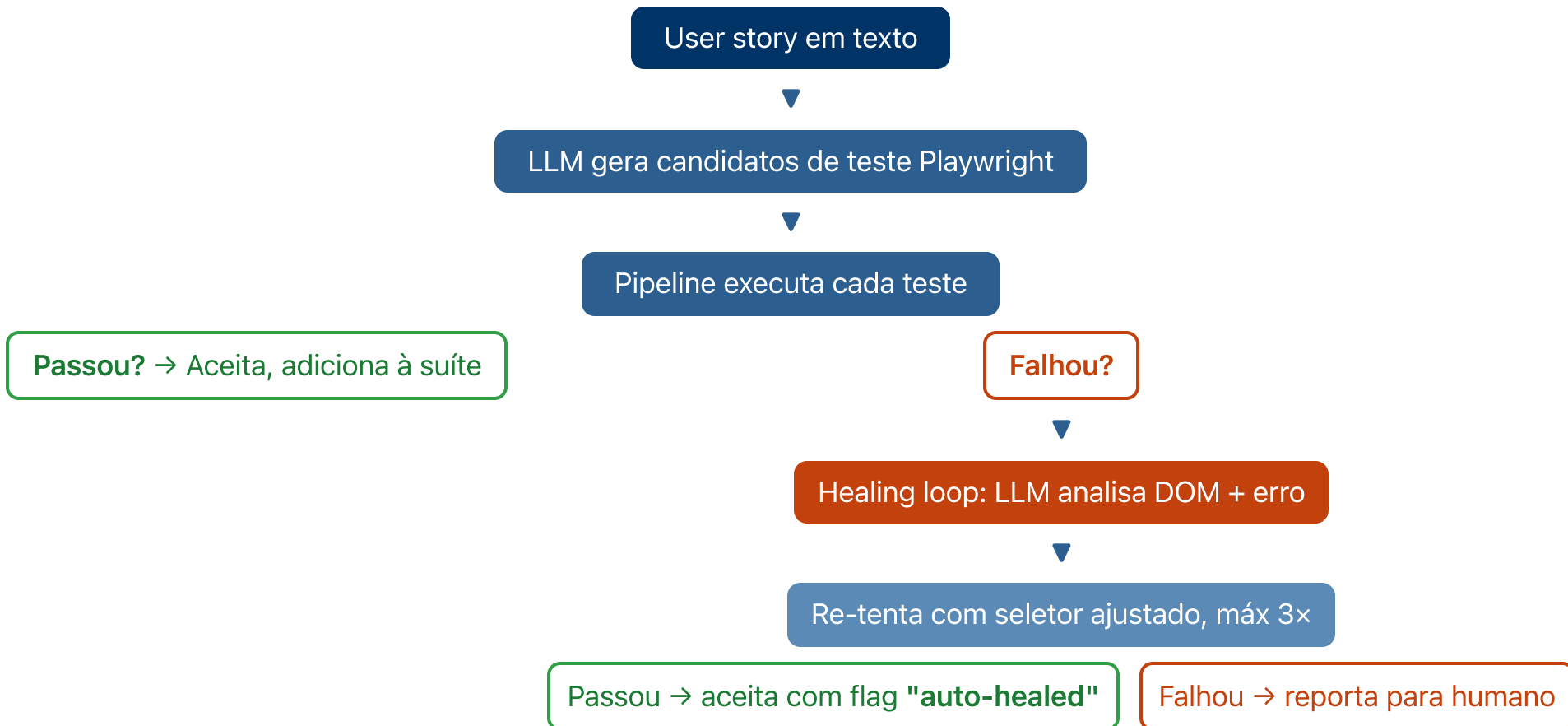
Por que IA reconfigura QA agora

Três avanços simultâneos em 2024–2026:

1. **LLMs com vision multimodal** (Claude, GPT-4o) → operam UI diretamente
2. **Computer Use API** (Anthropic) → agentes controlam browser via screenshot
3. **Visual AI ML-based** (Applitools) → reduz falsos positivos a quase zero

Isolados: úteis. **Combinados** → o que demorava semanas pode sair em horas.

Pipeline IA — desenho completo



Healing loop — implementação

```
async function generateAndHealTest(userStory: string): Promise<string> {
  let attempt = 0;
  let testCode = await generateTest(userStory); // API de LLM

  while (attempt < 3) {
    const result = await runTest(testCode);
    if (result.passed) return testCode;

    // Capturar DOM atual para contexto
    const dom = await captureCurrentDOM();

    // Pedir pro LLM corrigir com base no erro + DOM
    testCode = await client.messages.create({
      model: 'claude-opus-4-8',
      max_tokens: 2048,
      messages: [{
        role: 'user',
        content: `Teste falhou:\n${result.error}\n\nDOM atual:\n${dom}\n\nCorrigir seletor.`
      }],
    }).then(r => r.content[0].text);

    attempt++;
  }

  throw new Error('Falha após 3 tentativas. Revisão humana necessária.');
```

Self-Healing — Healenium

Biblioteca open source que age como proxy de seletor.

```
# Quando seletor quebra:  
# 1. Healenium captura DOM atual  
# 2. Busca elemento similar (heurística + ML)  
# 3. Atualiza seletor automaticamente  
# 4. Continua execução  
healenium:  
  recovery-tries: 3  
  similarity-threshold: 0.8  
  log-level: WARN
```

Risco crítico: seletor "curado" pode apontar para elemento **errado**.

Sempre auditar healing logs. Nunca confiar cego. Self-healing mascara refactors que quebrariam de propósito.

Parte 2 — Ecossistema de agentes

Synthetic data · Browser Use · Computer Use · trade-offs

Synthetic test data via LLM

```
const fixtures = await client.messages.create({
  model: 'claude-opus-4-8',
  messages: [{
    role: 'user',
    content: `Gere 20 fixtures de usuários brasileiros para teste:
- nome, email, telefone (formato BR), cpf (válido algoritmo)
- inclua edge cases: nomes longos, acentos, hifens, apóstrofes
- NUNCA use dados de pessoas reais – tudo fictício
- saída: JSON array`,
  }],
});
```

IA excelente em gerar **variedade realista com edge cases**. Sempre fictício — jamais dados reais de pessoas.

AI Agents — Browser Use (open source)

```
from browser_use import Agent
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-opus-4-8")

agent = Agent(
    task="""
    No site exemplo.com:
    1. Tente comprar um produto com cartão inválido
    2. Tente cupons inválidos antes do correto
    3. Reporte qualquer comportamento estranho ou mensagem confusa
    """,
    llm=llm,
)

result = await agent.run()
print(result)
```

Exploratory testing automatizado — encontra bugs não scriptados.

AI Agents — Anthropic Computer Use

API oficial para agentes que controlam desktop via screenshot + ações.

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-8",
    max_tokens=4096,
    tools=[{"type": "computer_20241022", "name": "computer"}],
    messages=[{
        "role": "user",
        "content": "Abra o app exemplo.com, faça login, e tente encontrar bugs visuais na tela de checkout."
    }]
)
```

Mais poderoso, mais caro, mais lento (~30s por ação). Ideal para exploratory profundo, não suíte de regressão.

Trade-offs honestos

Quando IA brilha:

- Transformação (story → estrutura de teste)
- Exploratory testing autônomo (Browser Use, Computer Use)
- Síntese de relatórios (Lighthouse, coverage, drift visual)
- Dados sintéticos com edge cases realistas

Quando IA falha:

- Definir o *que* assertar (oracle problem permanente)
- Decisão arquitetural de cobertura e estratégia
- Edge cases de domínio específico que LLM não conhece
- Custo de operação em escala (tokens × iterações)

Pitfalls comuns

- ✗ **Self-healing sem auditoria** — bugs reais ficam mascarados para sempre
- ✗ **Agente em loop infinito** — sempre limitar tentativas (máx 3–5)
- ✗ **Custo descontrolado** — `tracesSampleRate: 1.0` ou loop sem limite = custo alto
- ✗ **Aceitar testes gerados sem revisar asserções** — oracle problem
- ✗ **Usar Computer Use em CI** — lento demais (segundos por ação) para regressão rotineira

Resumo

- **Pipeline IA:** user story → gera testes → executa → healing loop em falha de seletor
- **Self-healing** (Healenium) útil mas perigoso — sempre auditar; não confiar cego
- **Synthetic data:** LLM gera fixtures realistas com edge cases; sempre fictício
- **Browser Use:** exploratory testing autônomo — bugs não scriptados
- **Computer Use:** poderoso mas caro — reservar para exploratory profundo
- **Regra de ouro:** IA escreve e explora; humano valida o oracle

Parte 3 — Prática guiada

Seu healing loop no lab IA

A prática — completar o pipeline

```
cd exercicios/03-lab-ia-test-gen/pratica  
ls src      # 01-gen-test.ts (pronto) · 02-healing-loop.ts (SEU) · 03-visual-diff.ts
```

- Na aula passada você **gerou** o teste; hoje você fecha o ciclo: **healing loop**
- `src/02-healing-loop.ts` tem os TODOs — o slide de implementação da Parte 1 é literalmente o que você vai escrever

Prática — o healing loop (02-healing-loop.ts)

```
# 1. sabote um seletor no spec gerado (ex.: heart → like)
# 2. rode o healing:
npm run heal -- generated/favoritar.spec.ts stories/favoritar.story.md
```

- TODO 1: rodar o spec — passou? nada a curar
- TODO 2: loop 1..3 : montar mensagem (código + erro + story) → LLM → gravar → re-rodar
- TODO 3: esgotou 3 tentativas → reportar pro humano (healing não é mágica)
- **Auditoria obrigatória:** o script imprime cada correção — o modelo consertou o seletor ou **enfraqueceu a asserção?** (anti-pattern da Parte 1)

Fechando o lab IA (entrega — 20 pts)

Entregável	Onde
Healing loop completo	<code>src/02-healing-loop.ts</code>
Visual diff completo (aula 3.5)	<code>src/03-visual-diff.ts</code>
Story SUA + spec gerado + log do healing	<code>stories/</code> + <code>generated/</code> + PR
Relatório crítico ~3 pgs (oracle, custo, limites)	<code>RELATORIO.md</code>

Rubrica completa no `enunciado.md` do lab. O relatório é o que separa "rodei um script" de "sei avaliar IA em QA" — capriche nele.

Como acompanhar

- **Screencast** — vídeo na sequência desta unidade
- **Travou na API?** — o pipeline imprime tokens/modelo; erros 401 = chave no `.env`
- **Office hours** — link no Canvas

Você chegou ao final da U3!

Boa sorte no projeto final.

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith